Dizayn patternləri və düzgün seçim

Design pattern təkrarlanan dizayn probleminin ümumi həll quruluşudur. Hazır köçürüləcək kod və ya məcburi kitabxana deyil. Pattern seçəndə əvvəl problemi, dəyişən hissəni və qəbul edilən əlavə mürəkkəbliyi izah etmək lazımdır. Bu dərs klassik 23 GoF pattern-in xəritəsini verir, backend-də tez rast gəlinənlərini nümunələrlə açır və əlavə arxitektura pattern-lərini ayrıca ayırır.

İlkin `Design Patterns.pptx` materialında Singleton, Mediator, factory ailəsi, Strategy və Adapter mövcud idi. Abstract Factory-nin “factory-lərin idarəçisi” kimi izahı, DI-nin runtime seçim edə bilmədiyi fikri və mediator-un hər dövrəni həll etdiyi iddiası burada dəqiqləşdirilir. Roadmap sənədi isə dərs planıdır; mövzunun tam izahını əvəz etmir.

Mündəricat

Dizayn patternləri və düzgün seçim **1**

1 Pattern dili və üç ailə . . . 2

2 Yaradılma patternlərinin xəritəsi . . . 2

3 Simple Factory Factory Method və Abstract Factory . . . 2

4 Builder Prototype və Singleton sərhədləri . . . 3

5 Struktur patternlərinin xəritəsi . . . 4

6 Adapter Decorator Proxy fərqi . . . 4

7 Davranış patternlərinin xəritəsi . . . 6

8 Strategy State və Template Method . . . 6

9 Mediator və Observer sərhədləri . . . 7

10 Repository Unit of Work CQRS və Outbox . . . 7

11 Seçim sualları və məşq . . . 7

12 Mənbələr və əlavə oxu . . . 8

1 Pattern dili və üç ailə

GoF, Gang of Four, klassik design patterns kitabının dörd müəllifinə verilən addır: Erich Gamma, Richard Helm, Ralph Johnson və John Vlissides. **Creational** pattern-lər yaradılmanı, **structural** pattern-lər obyektlərin quruluşunu, **behavioral** pattern-lər davranış və ünsiyyəti təşkil edir.

Interface istifadəçiyə verilən müqavilə, **implementation** onun konkret həllidir. **Composition** bir obyektin başqa obyektləri istifadə etməsi, **inheritance** sinfin başqa sinifdən törəməsidir. **Delegation** işi saxlanmış başqa obyektə ötürməkdir. **Client** pattern-dən istifadə edən kod, **context** isə cari davranışı saxlayıb işlədən obyekt ola bilər. Runtime proqramın işlədiyi vaxtdır.

Pattern-in dəyəri adı ilə deyil, dəyişiklik və asılılıq sərhədini necə qurması ilə ölçülür. Sadə bir funksiya problemi aydın həll edirsə əlavə pattern qatları vacib deyil. Eyni kod quruluşu fərqli niyyətlə başqa pattern adını ala bilər; niyyət önəmlidir.

2 Yaradılma patternlərinin xəritəsi

Pattern	Problem və konkret məna
Factory Method	Baza yaradıcı axın müəyyən edir, alt sinif yaratma metodunu əvəz edib hansı məhsulun yaradılacağını seçir. Məsələn, müxtəlif export formatları.
Abstract Factory	Bir-birinə uyğun məhsul ailəsini birlikdə yaradır. Məsələn, eyni mövzuya aid button və checkbox.
Builder	Mürəkkəb obyektin qurulmasını mərhələlərə ayırır. Məsələn, çox seçimli hesabat.
Prototype	Mövcud nümunədən yeni obyekt çıxarır. Kopyalanan daxili obyektlərin paylaşılması ayrıca həll edilməlidir.
Singleton	Seçilmiş kontekstdə bir nüsxəni və ona çıxışı təşkil edir. Paylaşılan vəziyyət və test çətinliyi riski daşıyır.

Product factory-nin yaratdığı obyekt üçün ümumi addır. **Creator** onun yaradılma nöqtəsini təqdim edir. **Clone** obyektin kopyasıdır. **Shallow copy** daxili istinadları da paylaşır; **deep copy** uyğun daxili obyektləri ayrıca kopyalayır. Hansı məlumatın paylaşılması təhlükəsizdirsə kopya qaydası ona görə seçilir.

3 Simple Factory Factory Method və Abstract Factory

Simple Factory bir metodda seçimə görə obyekt yarada bilər. Məsələn, `Create("pdf")` PDF exporter, `Create("csv")` CSV exporter qaytarır. Bu çox işlənən yanaşmadır, amma GoF Factory Method-un inheritance əsasında seçim nöqtəsi ilə eyni anlayış deyil. Hər `Create` adlı metod GoF Factory Method olmur.

C#

```
public interface IExporter
{
    string Export();
}

public abstract class ReportJob
{
    protected abstract IExporter CreateExporter();

    public string Run()
    {
        var exporter = CreateExporter();
        return exporter.Export();
    }
}

public sealed class CsvJob : ReportJob
{
    protected override IExporter CreateExporter()
        => new CsvExporter();
}

public sealed class CsvExporter : IExporter
{
    public string Export() => "Id,Total";
}
```

Bu Factory Method nümunəsində `Run` ümumi axındır, hansı exporter-in yaradılmasını alt sinif seçir. `abstract` baza sinfin həmin davranışı tamamlamadığını, `override` isə alt sinfin onu əvəz etdiyini göstərir. `protected` yalnız sinif və törəmələrinə görünən üzvdür. Nəticə sadələşdirilmiş mətn nümunəsidir, tam CSV fayl yazıcısı deyil. CSV vergüllə ayrılmış cədvəl mətn formatıdır.

Abstract Factory bir-birinə uyğun bir neçə məhsul verir: `IUiFactory.CreateButton()` və `CreateCheckbox()`. `DarkUiFactory` tünd mövzuya uyğun hər iki məhsulu yaradır. Məqsəd factory-ləri idarə etmək deyil, **related product family**, əlaqəli məhsul ailəsinin uyğunluğunu qorumaqdır. Yeni mövzu ailəsi əlavə etmək asandır, amma ailəyə yeni məhsul növü əlavə etmək bütün factory-lərə təsir edə bilər.

DI container yaradılmanı və lifetime-ı təşkil edir. Factory runtime seçimini ayrıca aydın edə bilər. Built-in DI factory delegate və keyed services kimi seçim imkanlarına malikdir. Ona görə “DI varsa factory lazım deyil” və “DI runtime seçim edə bilməz” cümlələrinin ikisi də həddən artıq ümumidir.

4 Builder Prototype və Singleton sərhədləri

Builder çox parametrli obyektin mərhələli qurulmasını idarə edir. Məsələn, `WithTitle`, `AddSection`, `Build`. **Fluent API** metodların obyekti geri qaytarıb zəncir yaratmasıdır. Hər fluent API Builder deyil; Builder-in niyyəti etibarlı mürəkkəb nəticə qurmaqdır. `Build` məcburi hissələrin verildiyini yoxlaya bilər. **Director** varsa qurulma ardıcıllığını idarə edən ayrıca obyektdir, hər builder-də məcburi deyil.

Prototype şablon obyekti kopyalayıb başlanğıc ayarlarını yenidən qurmaq xərcini azalda bilər. Amma sifariş nüsxəsində eyni `List<Item>` istinadını paylaşsan birinin dəyişməsi o birinə təsir edər. Deep copy də database connection və lock kimi resursların mənalı kopyasını avtomatik yaratmır.

Klassik Singleton çox vaxt `static Instance` ilə qlobal çıxış verir. **Static** üzv konkret obyektə deyil tipə bağlıdır. Gizli qlobal mutable state testlərin bir-birinə təsir etməsinə səbəb ola bilər. DI singleton lifetime isə bir provider daxilində xidmət nüsxəsinin paylaşılmasıdır. DI ilə açıq verilən singleton xidməti klassik gizli qlobal

çıxışla eyni dizayn deyil. Hər singleton avtomatik SRP pozuntusu sayılmır, amma paylaşılma və yaradılma məsuliyyətləri diqqətlə seçilməlidir.

Birdən çox tətbiq nüsxəsində hərəsinin öz singleton-u olur. Ümumi serverlərarası yeganə lider və ya paylaşılan məlumat lazımdırsa ayrıca koordinasiya tələb olunur. Thread-safe yaradılma obyektin bütün metodlarının thread-safe olması demək deyil. Thread safety paralel çağırışların düzgün nəticəni saxlamasıdır.

5 Struktur patternlərinin xəritəsi

Pattern	Problem və konkret məna
Adapter	Uyğun gəlməyən interfeysi client-in gözlədiyi müqaviləyə çevirir. Köhnə bank API-si üçün yeni ödəniş müqaviləsi nümunədir.
Bridge	Abstraksiya və implementasiyanın ayrıca dəyişməsinə imkan verir. Bildiriş növü ilə göndərmə kanalını ayırmaq olar.
Composite	Tək obyekt və obyekt ağacını eyni müqavilə ilə idarə edir. Fayl və qovluğun ümumi ölçü hesablaması nümunədir.
Decorator	Eyni müqaviləni saxlayan sarğı ilə əlavə davranış verir. Sorğuya log və ya keş əlavə etmək olar.
Facade	Mürəkkəb alt sistemə sadələşdirilmiş giriş təqdim edir. Sifariş üçün bir neçə addımı bir əməliyyatla əlaqələndirir.
Flyweight	Çox obyekt arasında dəyişməz ortaq məlumatı paylaşaraq yaddaş xərcini azaldır. Eyni şrift təsvirini paylaşmaq nümunədir.
Proxy	Başqa obyektə çıxışı nəzarətdə saxlayan əvəzləyici verir. Uzaq xidmət, giriş icazəsi və lazy yaradılma üçün istifadə oluna bilər.

Tree, ağac, valideyn uşaq əlaqəli quruluşdur. Composite-də yarpaq tək elementi, düyün isə başqa elementləri saxlayır. Flyweight-də **intrinsic state** paylaşılabilən daxili məlumat, **extrinsic state** isə çağırışa görə verilən xarici məlumatdır. Məsələn şriftin forması ortaq, hər simvolun ekrandakı yeri xaricidir.

6 Adapter Decorator Proxy fərqi

Adapter-in məqsədi müqaviləni uyğunlaşdırmaqdır. `LegacyBank.PayInCents(int cents)` köhnə metod, `IPayment.Pay(decimal amount)` yeni müqavilədirsə adapter məbləği qəpiyə çevirib köhnə metodu çağırır. Tək tip çevirmə bəs etmir: valyuta, yuvarlaqlaşdırma və xəta mənası da qorunmalıdır. **API** başqa kodun istifadə etdiyi giriş müqaviləsidir.

Decorator eyni müqaviləni saxlayıb davranış əlavə edir. Aşağıdakı nümunə metod çağırışını loglayır. Log işləmə barədə qeyddir.

C#

```
public interface IPrice
{
    decimal Get(int productId);
}

public sealed class LoggedPrice(IPrice inner) : IPrice
{
    public decimal Get(int productId)
    {
        Console.WriteLine($"Price requested: {productId}");
        return inner.Get(productId);
    }
}
```

`inner` sarınan əsas implementasiyadır. `$"...{...}"` string interpolation, yəni ifadənin dəyərini mətnə yerləşdirməkdir. Decorator zənciri ilə ölçmə, log və keş birləşdirilə bilər. Keş əvvəlki nəticəni müəyyən müddət saxlamaqdır. Keş nəticəni qaytarırsa məlumatın yenilənməsi və istifadəçiyə görə ayrılması da müqavilədə düşünülməlidir.

Proxy daha çox obyektə çatmağı idarə edir: məsələn, icazə yoxlayır və ya obyektin uzaqda olmasını gizlədir. Decorator əlavə davranış niyyəti daşıyır. Quruluşları oxşar ola bilər; seçilən ad niyyətə əsaslanır. Facade isə bir neçə alt sistemin istifadəsini sadələşdirə bilər və eyni interface-i saxlamağa məcbur deyil.

7 Davranış patternlərinin xəritəsi

Pattern	Problem və konkret məna
Chain of Responsibility	Sorğunu bir sıra işləyicilərdən keçirir; biri emal edə və ya növbətiyə ötürə bilər. Middleware buna yaxın quruluş nümunəsidir.
Command	Əməliyyat istəyini obyekt kimi saxlayır. Növbələmə, log və uyğun ssenaridə geri alma imkanı verir.
Interpreter	Kiçik dilin qrammatikasını obyektlərlə ifadə edib şərh edir. Sadə qayda dili nümunədir.
Iterator	Kolleksiyanın daxili quruluşunu açmadan elementləri gəzməyə imkan verir. foreach və IEnumerable bu ideyadan istifadə edir.
Mediator	Obyektlərarası qarşılıqlı əlaqəni vasitəçidə əlaqələndirir. Hər obyektin hamısını tanımasını azaldır.
Memento	Obyektin əvvəlki vəziyyətini onun daxilini yaymadan saxlayır və bərpa edir. Redaktorda undo nümunədir.
Observer	Hadisəyə abunə olmuş obyektlərə dəyişiklik bildirişi verir. C# event mexanizmi uyğun nümunədir.
State	Cari vəziyyətə görə davranışı ayrıca state obyektləri ilə dəyişir. Pending və Paid sifariş nümunədir.
Strategy	Eyni məqsəd üçün dəyişən alqoritmləri bir müqavilə altında təqdim edir. Endirim və çatdırılma hesabı nümunədir.
Template Method	Alqoritmin skeletini baza sinifdə saxlayır, bəzi addımları alt sinfə verir. Import axını nümunədir.
Visitor	Sabit obyekt quruluşuna yeni əməliyyatlar əlavə etməyi ayırır. Sintaksis ağacının müxtəlif analizləri nümunədir.

Grammar dil ifadələrinin qaydalarıdır. **Syntax tree** ifadənin hissələrini ağacda göstərən modeldir. **Undo** əvvəlki dəyişiklikdən geri dönməkdir. **Subscription** bildirişə abunəlikdir. Observer-də uzunömürlü publisher, hadisəni yayan obyekt, subscriber-i saxlayıb yaddaşda uzun müddət tuta bilər; abunəliyin bitirilməsi nəzərə alınmalıdır.

Command obyektinin mövcudluğu undo-nu avtomatik mümkün etmir. Xaricə göndərilmiş email və ya ödəniş ayrıca kompensasiya tələb edə bilər. Memento böyük vəziyyətin surətini saxlayırsa yaddaş xərci artır. Visitor yeni əməliyyat əlavə etməyi asanlaşdırır, amma yeni entity tipi əlavə etmək bütün visitor-ləri dəyişməyi tələb edə bilər.

8 Strategy State və Template Method

Strategy-də xarici seçim alqoritmi müəyyən edir: normal, express və ya kuryer çatdırılması. Context seçilmiş hesablamanı çağırır. State-də isə obyektin hazırkı biznes vəziyyəti əməliyyatın mümkünlüyünü və keçidləri müəyyən edir: sifariş Pending-dən Paid-a, Paid-dən Shipped-ə keçir. Shipped vəziyyətdə ödənişi yenidən qəbul etmək məntiqə uyğun olmaya bilər.

Transition, keçid, bir vəziyyətdən başqasına dəyişmədir. Strategy obyektləri adətən eyni məqsədli alternativ alqoritmlərdir. State obyektləri isə vəziyyət keçidlərini və həmin vəziyyətin qaydalarını modelləşdirir. Bu niyyət fərqi oxşar interface quruluşuna baxmayaraq vacibdir.

Template Method baza sinifdə “oxu, yoxla, çevir, saxla” ardıcıllığını sabitləşdirir. Alt sinif CSV və JSON oxuma addımını dəyişə bilər. Bu inheritance-a əsaslanır. Strategy isə davranışı ayrıca obyekt kimi verib composition istifadə edir. Alt siniflər çoxalır və addımlar müstəqil dəyişirsə composition daha çevik ola bilər.

9 Mediator və Observer sərhədləri

Mediator komponentlər arasında əlaqəni idarə edir: bir request uyğun handler-ə yönləndirilə bilər. **Handler** konkret əməliyyatı emal edən koddur. MediatR .NET kitabxanasıdır, Mediator pattern-in tərifi deyil. Kitabxana olmadan da pattern tətbiq edilir.

Mediator-a görə A və B artıq bir-birini constructor-da tanımaya bilər, amma A handler-i B request-i, B handler-i yenə A request-i göndərirsə runtime çağırış dövrəsi qalır. Vasitəçi əlavə etmək məsuliyyət bölgüsünü əvəz etmir. Mediator bütün biznes qaydalarını toplayan həddindən artıq böyük obyektə də çevrilməməlidir.

Observer-də publisher hadisəni elan edir, subscriber-lər reaksiya verir. Mediator isə əlaqənin necə koordinasiya olunacağını bilə bilər. Prosesdaxili event və paylanmış message broker eyni çatdırılma zəmanətinə malik deyil. **Message broker** mesajları proseslər və serverlər arasında daşıyan sistemdir. İnternet bağlantısı, təkrar çatdırılma və davamlı saxlama əlavə problemlərdir.

10 Repository Unit of Work CQRS və Outbox

Bunlar klassik 23 GoF pattern siyahısının hissəsi deyil, amma backend müsahibəsində vacibdir. **Repository** domain obyektlərinə kolleksiya kimi çıxış verərək saxlanma sorğularını bir sərhəddə toplayır. Domain biznes anlayışları və qaydalarıdır. **Unit of Work** bir iş ərzində dəyişən obyektləri izləyib yazmaları əlaqələndirir. EF Core DbContext bu xüsusiyyətlərin bir çoxunu artıq verir. Üzərinə əlavə generic repository yazmaq hər layihədə məcburi deyil.

CQRS, Command Query Responsibility Segregation, dəyişiklik əmrləri ilə oxuma sorğularının model və məsuliyyətlərini ayırır. Eyni bazadan istifadə edə bilər. Ayrı baza, mikroservis və event sourcing məcburi şərt deyil. **Event sourcing** cari vəziyyəti hadisələr tarixindən qurmaq yanaşmasıdır və CQRS ilə eyni anlayış deyil.

Outbox baza dəyişikliyi ilə göndəriləcək hadisəni eyni transaction-da saxlayır. Ayrı worker sonradan hadisəni broker-ə göndərir. Beləcə “sifariş saxlandı, amma hadisə itdi” pəncərəsi daralır. Ancaq hadisə təkrar göndərilə bilər. **Idempotency** eyni əməliyyat təkrar gələndə əlavə yanlış təsir yaratmamaqdır; məsələn, eyni hadisə ID-si ilə iki dəfə kredit yazmamaq. Sadə “SaveChanges et, sonra Publish et” iki müstəqil sistem arasında atomiklik yaratmır.

Retry keçici uğursuzluqda təkrar cəhddir. **Timeout** gözləmə müddətini məhdudlaşdırır. **Circuit breaker** davamlı xəta alan asılılığa çağırışları müvəqqəti dayandırır. **Bulkhead** bir hissənin bütün resursları tutmaması üçün paralel işi məhdudlaşdırır. Bunlar GoF ailəsindən ayrı əməliyyat və etibarlılıq pattern-ləridir. Ödənişi kor retry etmək iki dəfə tutmaya səbəb ola bilər; idempotency ilə birlikdə düşünülməlidir.

11 Seçim sualları və məşq

Ödəniş kitabxanasının metodu uyğun gəlmir? Adapter. Müqavilə və xəta mənası çevrilir.

Eyni metoda keş əlavə etmək istəyirsən? Decorator uyğun ola bilər. Müddət, istifadəçi sərhədi və invalidation, yəni köhnə keşin silinməsi, müəyyən edilir.

Alqoritmi dəyişmək istəyirsən? Strategy. Obyektin biznes vəziyyəti davranışı dəyişirsə State-i düşün.

Bir-birinə uyğun bir neçə məhsul yaranmalıdır? Abstract Factory. Bir create metodunda seçim etmək kifayətdirsə Simple Factory də yetə bilər.

Məşq. Çatdırılma qiyməti üçün iki Strategy yaz. Xarici kuryer API-sini Adapter ilə həmin müqaviləyə uyğunlaşdır. Çağırış sayını Decorator ilə qeyd et. Sonra hər pattern-i çıxarsan nə çətinləşəcəyini yaz. Cavab “pattern adı azaldı” deyil, konkret dəyişiklik və müqavilə ilə izah olunmalıdır.

12 Mənbələr və əlavə oxu

- Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides - Design Patterns kitabı. Klassik 23 pattern kataloqu üçün əsas əsərdir; burada kitabdan uzun sitat istifadə edilmir.
- Martin Fowler - Repository
- Martin Fowler - Unit of Work
- Martin Fowler - CQRS
- Microsoft Azure Architecture Center - Cloud design patterns

Veb mənbələr 28 oktyabr 2026 tarixində yoxlanılıb. C# nümunələri pattern-in əlaqələrini göstərir, real ödəniş və mesaj göndərmə əməliyyatı icra etmir.